

Individual Contribution Report for Project 3

Name: Berat Sayın

Student ID: 2019400261

Group Number: 51

Tasks & Contributions:

For this project, my main responsibility was the Index and Query Layer of the modular DBMS engine. I worked primarily on the *file_index_manager* module, including the File and Index Manager, heap scan implementation, static hash index, B+ tree index, system catalog, and slotted page format. In this layer, I implemented record creation, deletion, primary-key search, range search, duplicate primary-key checks, and index maintenance during insert and delete operations.

I also worked on the *query_processor* module. This included parsing input commands, dispatching operations to the File and Index Manager, writing query results to *output.txt*, logging all operations to *log.csv*, and generating statistics output through the *stats* and *explain* commands. In addition, I contributed to *archive.py*, which wires all layers together, and *workload_generator.py*, which generates workloads for testing and experiments.

I prepared the full implementation explanation video content and the README.md file. The explanation video covered the overall architecture, each implemented layer, the page and record format, indexing strategies, buffer interaction, query execution, and statistics handling.

Collaboration & Challenges:

During the project, my partner and I collaborated closely on the system design. Before implementation, we discussed the general architecture, how the layers should interact, and how the configuration options should affect the behavior of the system. After implementing our own parts, we reviewed each other's code to catch design issues, bugs, and inconsistencies between modules.

One of the main challenges was making sure that all layers worked correctly together while still keeping the modular structure clear. Index maintenance was also challenging, especially ensuring that the hash index and B+ tree stayed consistent after insertions and deletions. Another challenge was testing different configurations, because the same input file

had to work correctly with heap scan, hash index, and B+ tree strategies. At the end, we tested the system together using both small manual test cases and generated workloads to verify correctness and compare behavior across configurations.

Self-Assessment:

I believe I made a strong contribution to the project, especially in the query processing and indexing parts. This project helped me better understand how a DBMS is organized internally, from parsing commands at the top layer to locating records through heap scans or indexes at the lower layers. Implementing the slotted page structure, catalog, hash index, and B+ tree gave me practical experience with concepts that were previously mostly theoretical.

I also improved my debugging and testing skills. Many bugs only became visible when different modules were connected together, so I learned the importance of testing the full system instead of only testing individual functions. If I had more time, I would improve the strictness of the layer interfaces even further and add more automated tests for edge cases and persistence across multiple runs.

Use of AI Tools

I used Codex and Gemini-CLI as support tools during the project. While fixing mistakes or implementing new features, I asked AI tools for help to reason about whether my code would work correctly and to identify possible edge cases. I also used AI assistance to review parts of the code and help find incorrect logic that I might have missed.